CSC-239 *Java Programming*

Chapter 11: Inheritance and Polymorphism (Part 1)

This lecture is being recorded.

October 15, 2025

NOTICE to Students: Portions of these lecture slides are
Copyright 2021 Pearson Education, Inc.
We have a license to use this material ***only*** in the context of this course.
There is NO PERMISSION to share these lecture slides with anyone not taking the course.
There is NO PERMISSION to post these lecture slides on the Internet.

Reminder: Project 1 due October 15

- Project 1 is due TODAY: Wednesday, October 15.
- As stated in the *Syllabus*, the grade penalty for turning in a project late is:

  1% per day (after a 1-day grace period), up to a maximum grade penalty of 20%.

  Any project that is 21 days late (or more) receives a grade of ZERO.
- What this means is: if you need a <u>few</u> extra days to do a good job on your project, then the late penalty will not be very much.
- However, if you need significantly more time than that, you should meet with the instructor to get some help.

Online Information

- Java Tutorials:

  https://docs.oracle.com/javase/tutorial/

- Java API:

  https://docs.oracle.com/en/java/javase/21/docs/api/index.html

- The official "*Grader Than*" documentation:

  https://docs.graderthan.com/student/

- *Grader Than* Support E-mail:

  support@graderthan.com

Motivations

- Suppose you need to define classes to model circles, rectangles, and triangles.
- These classes have many common features.
- What is the best way to design these classes to avoid lots of duplicated code?
- The answer is to use *inheritance*.

Superclasses and Subclasses (1)

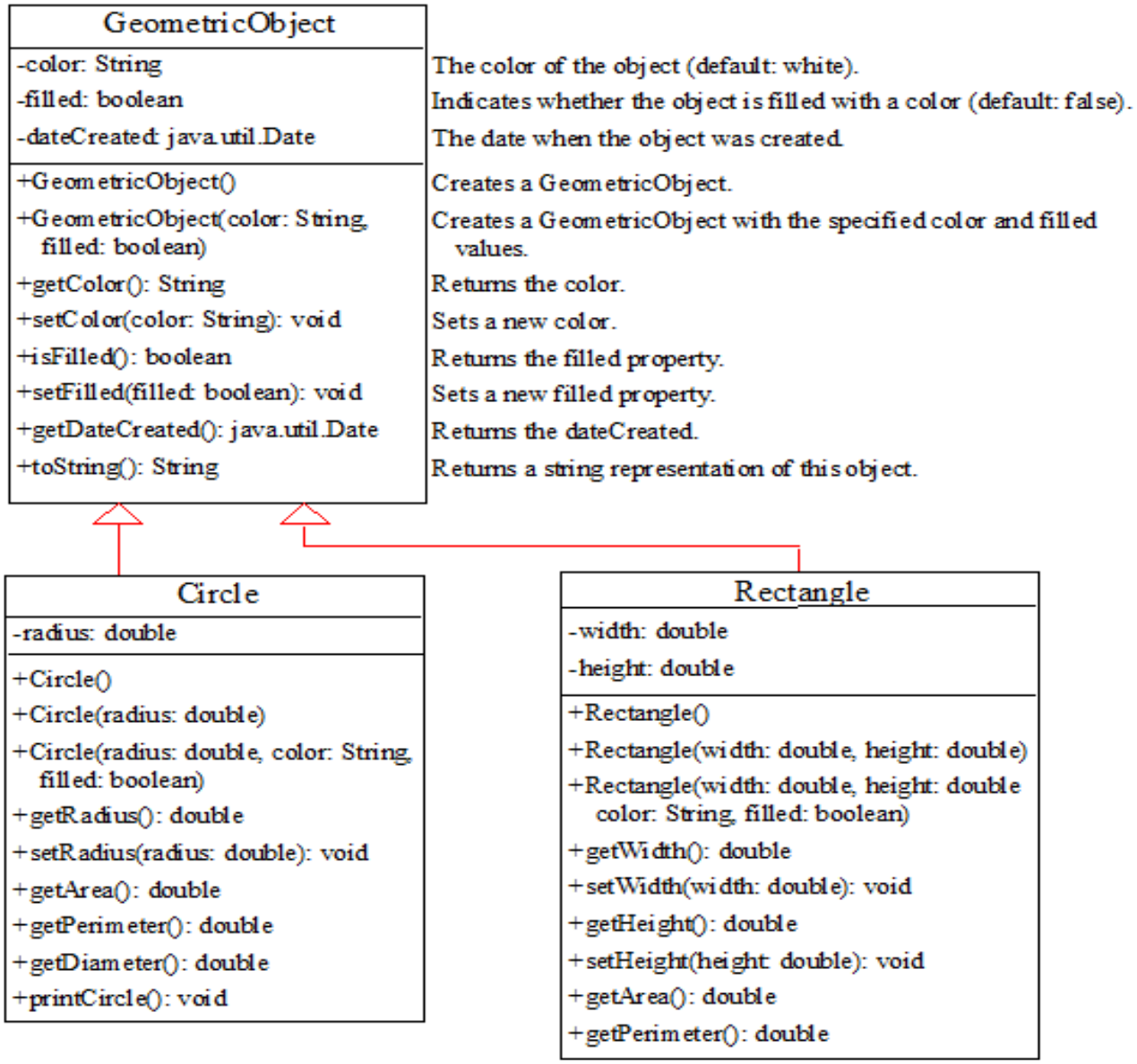
- Inheritance involves one class being a *subclass* of another class.
- The UML diagram on the next slide illustrates an inheritance relationship among the

  GeometricObject

  Circle, and

  Rectangle classes.
- These classes are described in Chapter 11 of the textbook.
- The hollow arrow showing inheritance in a UML diagram always points to the superclass.

Superclasses and Subclasses (2)



Inheritance Syntax

```
public class SuperClass {
    . . .
}

class SubClass extends SuperClass {
    . . .
}
```

The *IsA* Relationship

- An object of the subclass **IS** an object of the superclass.
- ***Superclass*** is also referred to as the
  - Parent class
  - Base class
- ***Subclass*** is also referred to as the
  - Child class
  - Extended class
  - Derived class

CAUTION: Do not confuse the ***IsA*** relationship with the ***HasA*** relationship (see Chapter 10).

Code Reuse

- The child class inherits all accessible data fields and methods from the parent class.

  No need for the child class to create duplicates of methods from the parent class, if they can be re-used.

- The child class is usually a more specialized class than the parent class:
  - A **Circle** is a more specialized **GeometricObject**.
  - A **Rectangle** is a more specialized **GeometricObject**.

Inheritance Chain

Suppose we have three classes --

Person, Employee, Faculty:

Employee extends **Person**.

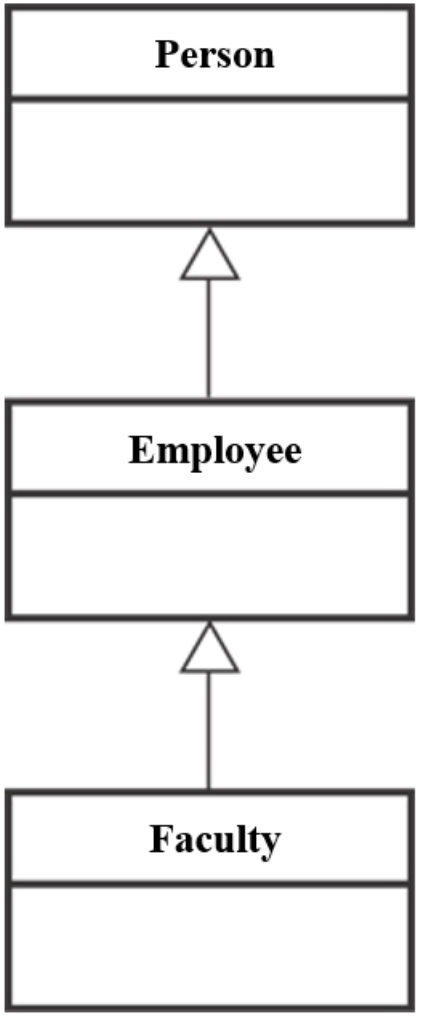
Faculty extends **Employee**.

A **Faculty** object is an **Employee** object.

An **Employee** object is a **Person** object.

Therefore:

A **Faculty** object is a **Person** object.



Superclass Constructor(s)

- Are the Constructors inherited from a superclass to a subclass?
- No, they are not ***inherited*** by the SubClass, but they are invoked
  - Either explicitly: using the **`super`** keyword.
  - Or implicitly:

    If the keyword **`super`** *is not explicitly used, the superclass's* ***no-arg*** *constructor is automatically invoked.*

Some Superclass Constructor Is *Always* Invoked

- A subclass constructor may invoke another (overloaded) constructor in its own class:

```
this();
```

- or, a subclass may explicitly invoke its superclass's constructor:

```
super();
```

- If none of them is invoked explicitly, the compiler puts `super();` as the first statement in the subclass constructor.

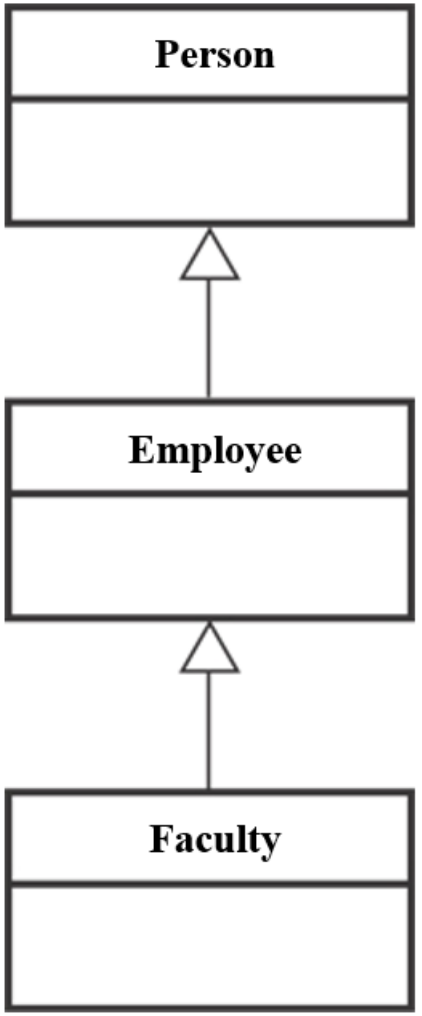
Remember the *IsA* Relationship

A **Faculty** object *is* an **Employee** object.

An **Employee** object *is* a **Person** object.

Therefore:

A **Faculty** object *is* a **Person** object.



Constructor Chaining Example

```
public class Faculty extends Employee {
      public static void main(String[] args) {
            new Faculty();
      }

      public Faculty() {
           System.out.println("(4) Faculty's no-arg constructor");
     }
}

class Employee extends Person {
      public Employee() {
            this("(2) Invoke Employee's overloaded constructor");
            System.out.println("(3) Employee's no-arg constructor");
      }

      public Employee(String s) {
             System.out.println(s);
      }
}

class Person {
      public Person() {
             System.out.println("(1) Person's no-arg constructor");
      }
}
```

Constructor Chaining Example: Output

See also sample code (on Moodle):

Ch11_sample_code_ConstructorChaining

(1) Person's no-arg constructor is invoked
(2) Invoke Employee's overloaded constructor
(3) Employee's no-arg constructor is invoked
(4) Faculty's no-arg constructor is invoked

Problem: A Superclass without a no-arg Constructor

```
public class Apple extends Fruit {
  public static void main(String[] args) {
     Apple myApple = new Apple();
  }
}

class Fruit {
  public Fruit(String name) {
    System.out.println("Fruit's constructor");
  }
}
```

- This results in a compile error.

Same Superclass with a no-arg Constructor

```
public class Apple extends Fruit {
  public static void main(String[] args) {
     Apple myApple = new Apple();
  }
}

class Fruit {
  public Fruit(String name) {
     System.out.println("Fruit's constructor");
  }
  public Fruit() {
     System.out.println("no-arg constructor");
  }
}
```

- This works.